

Homework 07

Rahaf Alabdullah (rahafaa2)

Question 1: LDA

```
import numpy as np
from pathlib import Path
import pandas as pd
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import confusion_matrix

data_dir = Path(__file__).parent / "data"
heart = pd.read_csv(data_dir / "saheart.csv")
heart["famhist"] = heart["famhist"].map({"Absent": 1, "Present": 2})
X = heart.drop(columns="chd").to_numpy()
y = heart["chd"].to_numpy()

n = X.shape[0]
K = len(np.unique(y))
pi_K = np.array([(y == k).mean() for k in np.unique(y)])
mu_K = np.array([X[y == k].mean(axis=0) for k in np.unique(y)])
X_centered = [X[y == k] - mu_K[k] for k in np.unique(y)]
Sigma = sum([Xc.T @ Xc for Xc in X_centered]) / (n - K)
print(pd.DataFrame(pi_K, columns=["pi_k"]).round(3))
print(pd.DataFrame(mu_K, columns=heart.drop(columns="chd").columns).round(3))
print(pd.DataFrame(Sigma, columns=heart.drop(columns="chd").columns, index=heart.drop(columns="chd").columns).round(3))

Sigma_inv_mu = np.linalg.solve(Sigma, mu_K.T)
first_term = X @ Sigma_inv_mu
second_term = np.diag(mu_K @ Sigma_inv_mu)
score = first_term - 0.5 * second_term + np.log(pi_K)
y_hat = np.argmax(score, axis=1)
confusion_matrix_manual = confusion_matrix(y, y_hat)
print(pd.crosstab(y, y_hat, rownames=["Observed"], colnames=["Predicted"]))
print(f"Dimension check: X {X.shape}, Sigma_inv {np.linalg.inv(Sigma).shape}, mu_K {mu_K.shape}, score {score.shape}")

LDA_model = LinearDiscriminantAnalysis(solver="svd").fit(X, y)
LDA_predictions = LDA_model.predict(X)
confusion_matrix_LDA = confusion_matrix(y, LDA_predictions)
agreement = np.array_equal(y_hat, LDA_predictions)
print(pd.crosstab(y, LDA_predictions, rownames=["Observed"], colnames=["Predicted"]))
print(f"Do the confusion matrices match? {agreement}")
```

(a) Prior probabilities, class means and pooled covariance matrix (denominator $n - K$).

Class	Prior								
chd = 0	0.654								
chd = 1	0.346								

Class	sbp	tobacco	ldl	adiposity	famhist	typea	obesity	alcohol	age
chd = 0	135.460	2.635	4.344	23.969	1.318	52.368	25.737	15.931	38.854
chd = 1	143.738	5.525	5.488	28.120	1.600	54.494	26.623	19.145	50.294

	sbp	tobacco	ldl	adiposity	famhist	typea	obesity	alcohol	age
sbp	405.435	14.585	4.581	49.164	0.337	-15.588	18.939	64.400	95.134
tobacco	14.585	19.243	0.763	7.538	0.016	-2.057	1.833	20.517	22.765
ldl	4.581	0.763	4.001	6.033	0.092	0.345	2.660	-2.533	6.479
adiposity	49.164	7.538	6.033	56.753	0.433	-5.310	22.708	16.119	60.509

	sbp	tobacco	ldl	adiposity	famhist	typea	obesity	alcohol	age
famhist	0.337	0.016	0.092	0.433	0.226	0.081	0.184	0.768	0.997
typea	-15.588	-2.057	0.345	-5.310	0.081	95.566	2.640	7.960	-20.278
obesity	18.939	1.833	2.660	22.708	0.184	2.640	17.615	4.689	15.697
alcohol	64.400	20.517	-2.533	16.119	0.768	7.960	4.689	598.277	27.886
age	95.134	22.765	6.479	60.509	0.997	-20.278	15.697	27.886	184.132

(b) In-sample fitted labels from the scores computed by hand. Dimensions: $\mathbf{X}$ is 462×9 , $\hat{\Sigma}^{-1}$ is 9×9 , the class means are 2×9 and the score matrix is 462×2 .

Observed \ Predicted	0	1
0	258	44
1	73	87

(c) `LinearDiscriminantAnalysis(solver="svd")` with empirical priors:

Observed \ Predicted	0	1
0	258	44
1	73	87

Yes, the results match: all 462 predicted labels agree. This is expected because `LinearDiscriminantAnalysis` fits the same model as part (b). It uses the same empirical priors, class means, and pooled covariance, and it assigns each observation to the class with the largest linear discriminant score. The only difference is computational: in part (b) we calculated each step ourselves, while the `svd` solver gets the same result without inverting Σ directly.

Question 2: QDA and marginal screening

```
import numpy as np
from pathlib import Path
import pandas as pd
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.metrics import confusion_matrix

data_dir = Path(__file__).parent / "data"
zip_train = pd.read_csv(data_dir / "zip-train.csv.gz", header=None)
zip_test = pd.read_csv(data_dir / "zip-test.csv.gz", header=None)
y_train = zip_train.iloc[:, 0].to_numpy()
X_train = zip_train.iloc[:, 1:].to_numpy()
y_test = zip_test.iloc[:, 0].to_numpy()
X_test = zip_test.iloc[:, 1:].to_numpy()
print(f"Train shape: {X_train.shape}, Test shape: {X_test.shape}")

def largest_variance_indices(X, m):
    pixel_variance = X.var(axis=0)
    return np.argsort(pixel_variance)[-m:]

m_part_a=40
m_part_c=10
largest_40_indices = largest_variance_indices(X_train, m_part_a)
largest_10_indices = largest_variance_indices(X_train, m_part_c)
X_train_40 = X_train[:, largest_40_indices]
X_test_40 = X_test[:, largest_40_indices]
X_train_10 = X_train[:, largest_10_indices]
X_test_10 = X_test[:, largest_10_indices]

QDA_40 = QuadraticDiscriminantAnalysis(tol=1e-8).fit(X_train_40, y_train)
QDA_10 = QuadraticDiscriminantAnalysis(tol=1e-8).fit(X_train_10, y_train)
QDA_40_predictions = QDA_40.predict(X_test_40)
QDA_10_predictions = QDA_10.predict(X_test_10)
confusion_matrix_40 = confusion_matrix(y_test, QDA_40_predictions)
confusion_matrix_10 = confusion_matrix(y_test, QDA_10_predictions)
print("confusion matrix for m=40:")
print(pd.crosstab(y_test, QDA_40_predictions, rownames=["Observed"], colnames=["Predicted"]))
print("confusion matrix for m=10:")
print(pd.crosstab(y_test, QDA_10_predictions, rownames=["Observed"], colnames=["Predicted"]))
print(f"Test misclassification rate (40 pixels): {np.mean(QDA_40_predictions != y_test):.3f}")
print(f"Test misclassification rate (10 pixels): {np.mean(QDA_10_predictions != y_test):.3f}")

print(pd.DataFrame({"40 pixels": [np.mean(QDA_40_predictions != y_test)], "10 pixels": [np.mean(QDA_10_predictions != y_test)]},
index=["Test misclassification rate"]).round(3))
```

(a) Test confusion matrix, 40 highest-variance pixels:

Observed \ Predicted	0	1	2	3	4	5	6	7	8	9
0	319	0	24	6	2	2	1	0	5	0
1	0	238	5	0	11	0	7	0	3	0
2	8	0	168	8	8	3	0	0	3	0
3	5	0	4	128	0	14	0	0	15	0
4	0	2	13	1	150	1	0	1	7	25
5	12	0	3	5	1	131	0	0	6	2
6	0	0	8	0	1	8	148	0	5	0
7	1	0	6	1	6	0	0	124	2	7
8	1	0	10	12	3	2	0	0	137	1
9	0	0	1	2	7	1	1	4	9	152

Test misclassification rate (40 pixels): **0.155**

(b) Yes, QDA works with 40 pixels. In the lecture, each class had fewer images than the 256 predictors, so the class covariance matrices were singular. Here even the smallest class (542 images) is much larger than $p = 40$, so every class covariance matrix is full rank and invertible.

(c) Test confusion matrix, 10 highest-variance pixels:

Observed \ Predicted	0	1	2	3	4	5	6	7	8	9
0	298	0	11	1	1	16	28	4	0	0
1	0	241	10	0	3	1	1	0	2	6
2	19	2	95	14	14	3	22	21	5	3
3	5	0	6	103	3	42	0	0	7	0
4	1	5	14	0	95	2	4	15	17	47
5	15	0	6	25	4	96	4	2	8	0
6	41	0	15	4	0	18	78	7	5	2
7	4	1	6	0	6	0	1	122	0	7
8	1	2	11	36	12	11	4	1	74	14
9	0	2	9	2	20	0	0	16	10	118
		40 pixels		10 pixels						
Test misclassification rate		0.155		0.342						

With 10 pixels the test error rises to 34.2%. Fewer pixels mean fewer covariance parameters (55 per class), which lowers variance, but 10 pixels cannot capture digit shapes, which raises bias. Similar-looking digits are confused more often (e.g., 6 as 0, 3 as 5, 4 as 9). Here the increase in bias outweighs the reduction in variance.

Question 3: Compare LDA and QDA

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.discriminant_analysis import (LinearDiscriminantAnalysis,
                                           QuadraticDiscriminantAnalysis)

seed= 4323
rng = np.random.default_rng(seed)
rhos = [0, 0.2, 0.4, 0.6, 0.8, 0.9]
p = 10
n_train = 100
n_test = 1000
n_reps = 50
print(f"Random seed: {seed}")

def simulate(n_per_class, rho, rng):
    # Class 1
    mean1 = [-1, 0]
    cov1 = [[1, rho], [rho, 1]]
    X1 = rng.multivariate_normal(mean1, cov1, size=n_per_class)
    noise1 = rng.standard_normal((n_per_class, 8))
    X_class1 = np.hstack((X1, noise1))

    # Class 2
    mean2 = [1, 0]
    cov2 = [[1, -rho], [-rho, 1]]
    X2 = rng.multivariate_normal(mean2, cov2, size=n_per_class)
    noise2 = rng.standard_normal((n_per_class, 8))
    X_class2 = np.hstack((X2, noise2))

    # Combine classes
    X = np.vstack((X_class1, X_class2))
    y = np.repeat([1, 2], n_per_class)

    return X, y

def fit_and_score(X_train, y_train, X_test, y_test):
    # Fit LDA
    lda_model = LinearDiscriminantAnalysis(priors=[0.5, 0.5])
    lda_model.fit(X_train, y_train)
    lda_predictions = lda_model.predict(X_test)
    lda_error = np.mean(lda_predictions != y_test)

    # Fit QDA
    qda_model = QuadraticDiscriminantAnalysis(priors=[0.5, 0.5])
    qda_model.fit(X_train, y_train)
    qda_predictions = qda_model.predict(X_test)
    qda_error = np.mean(qda_predictions != y_test)

    return lda_error, qda_error

results = np.empty((len(rhos), 3))
for i, value in enumerate(rhos):
    X_train, y_train = simulate(n_train, value, rng)
    X_test, y_test = simulate(n_test, value, rng)
    lda_error, qda_error = fit_and_score(X_train, y_train, X_test, y_test)
    results[i] = [value, lda_error, qda_error]
results_df = pd.DataFrame(results, columns=["rho", "LDA error", "QDA error"])
print(results_df.round(3))

results_reps = np.empty((len(rhos), n_reps, 2))
for i, value in enumerate(rhos):
    for j in range(n_reps):
        X_train, y_train = simulate(n_train, value, rng)
        X_test, y_test = simulate(n_test, value, rng)
        lda_error, qda_error = fit_and_score(X_train, y_train, X_test, y_test)
        results_reps[i, j] = [lda_error, qda_error]
average_errors = results_reps.mean(axis=1)
```

```
plt.figure(figsize=(8, 6))
plt.plot(rhos, average_errors[:, 0], marker="o", label="LDA")
plt.plot(rhos, average_errors[:, 1], marker="o", label="QDA")
plt.xlabel("Correlation (rho)")
plt.ylabel("Average Test Error")
plt.title("Average Test Error vs. Correlation for LDA and QDA")
plt.legend()
plt.tight_layout()
plt.show()
average_errors_df = pd.DataFrame(average_errors, columns=["LDA error", "QDA error"])
average_errors_df.insert(0, "rho", rhos)
print(average_errors_df.round(3))
```

Random seed: 4323.

(a) Test misclassification rates from one dataset per ρ :

rho	LDA error	QDA error
0.0	0.167	0.210
0.2	0.169	0.208
0.4	0.165	0.190
0.6	0.162	0.171
0.8	0.162	0.135
0.9	0.175	0.092

(b) Average test misclassification rates over 50 repetitions:

rho	LDA error	QDA error
0.0	0.172	0.197
0.2	0.171	0.198
0.4	0.172	0.192
0.6	0.171	0.174
0.8	0.169	0.138
0.9	0.168	0.100

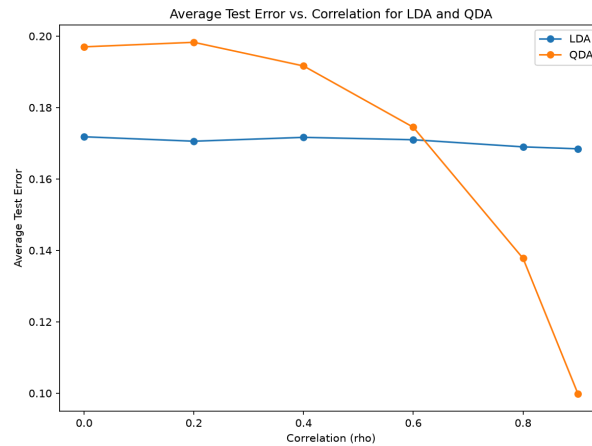


Figure 1: Average test error against ρ for LDA and QDA (50 repetitions)

(c) For small ρ , LDA has lower test error than QDA (about 0.17 vs 0.20 at $\rho = 0$). The two classes have (nearly) the same covariance matrix, so LDA's shared-covariance assumption is correct and it estimates only one covariance matrix. QDA estimates a separate matrix for each class, which adds variance without reducing bias. As ρ increases, the classes keep the same variances but their correlations point in opposite directions, so the best boundary is curved. LDA's linear boundary becomes biased and its error stays around 0.17, while QDA can use the difference in correlation and its error drops to about 0.10 at $\rho = 0.9$. The curves cross between $\rho = 0.6$ and 0.8, where QDA's reduction in bias starts to outweigh its extra variance. The eight noise covariates carry no information about the class, but both methods must still estimate their means, variances and covariances. This adds variance and raises test error, and it hurts QDA more because it estimates these covariances separately for each class.

Question 4: Create and use an LDA/QDA analysis skill

(a) Complete contents of SKILL.md (4,314 bytes):

```
---
name: lda-qda-report
description: Run a cross-validated LDA/QDA classification analysis on a given dataset and write a one-page statistical report.
disable-model-invocation: true
argument-hint: "<data file> <response> <predictors and coding> [seed]"
---
```

LDA/QDA Analysis Report

Run this skill only when the user invokes it explicitly. The user supplies the dataset, response, predictors (with any coding), and optionally a seed and output folder. If any of these are missing, ask before starting. Never assume dataset-specific facts; base every statement on the results you compute.

Tools

Python with numpy, pandas, scikit-learn and matplotlib. Write one script that reproduces every number, figure and table in the report.

Analysis rules

1. **Load and check.** Report n, number of predictors, class counts, and any missing values. Do not drop or recode variables beyond what the user asks.
2. **Reserve a test set first.** Make a stratified 30% test split with the seed before any exploration, preprocessing or model choice. Do not look at the test set again until step 6.
3. **Fixed CV folds.** Create one stratified 5-fold split of the training data with the seed and reuse the same folds for every candidate and setting.
4. **Candidates.** Every classifier must be LDA, QDA, or a combination:
 - LDA and QDA on all predictors (always included as baselines);
 - standardize + PCA with d components + LDA or QDA, d from 1 to p;
 - blend: $P = w * P_{LDA} + (1 - w) * P_{QDA}$, w in {0, 0.1, ..., 1}, predict the class with the largest P.Use empirical class priors. Put every preprocessing step (scaling, PCA) inside a scikit-learn Pipeline so it is fitted on the training folds only.
5. **Select.** Compute the mean CV misclassification rate and its standard error ($SD / \sqrt{5}$) for each candidate. Choose the lowest mean error; if a simpler candidate (LDA before QDA, fewer components) is within one SE, prefer it and say so.
6. **Final test.** Refit the selected model with its preprocessing on the full training set. Evaluate it once on the test set: misclassification rate and confusion matrix (observed rows, predicted columns). Also report the test error of plain LDA and QDA for reference only; do not use these numbers to change the selection.
7. If a fit fails or warns (e.g. singular covariance), record it, drop that candidate, and mention it in the limitations.

Report

Write a Quarto document (`.qmd`) that renders to a PDF of **at most one printed page** (letter size, 11pt, 0.75in margins). Hide code; show only results. Render it and confirm the page count; shorten text, not results, if it is too long. Sections, in order:

1. **Title** naming the dataset and response, plus one line with n, p, the seed and the split.
2. **Methods** (3-4 sentences): candidates, the train/test split, the 5-fold CV, and the selection rule.
3. **Figure**: CV misclassification rate (mean +/- 1 SE) for each candidate family against its tuning setting (PCA components or blend weight), with plain LDA and QDA as horizontal reference lines. Mark the selected model.
4. **Table**: one row per candidate family at its best setting: model, setting, CV error, SE. Bold the selected model. Rates to 3 decimals.
5. **Results** (2-3 sentences): selected model and setting, its test misclassification rate, and the confusion matrix as a small inline table.
6. **Findings and limitations** (3-5 bullets): what the comparison suggests about the class covariance structure, how much the extensions helped over

```

plain LDA/QDA, the test set size and its uncertainty, and any failed fits.

## Figure and table style

- Figure 6.5 x 2.8 in, 150 DPI, white background, no top or right spines.
- Colorblind-friendly palette, one color per model family, consistent with
  the table. Axis labels in sentence case with units, 9-10pt font.
- Legend inside the plot area or below it, never covering data.
- Table with a header row and no vertical lines; numbers right-aligned.

## Final check before returning

- The test set was used exactly once, after selection.
- Every number in the text matches the script output.
- The PDF is one page, and the seed is reported.
- Summarize the selected model and test error for the user in two lines.

```

(b) The skill was applied to the SAheart data from Question 1. The one-page analysis report it produced is on the next page.

SAheart: LDA/QDA Classification of chd

$n = 462$ (302 without, 160 with CHD), $p = 9$ numeric predictors (`famhist` coded 1/2) · seed 4323 · stratified 70/30 split: 323 training, 139 test.

Methods. 31 candidates: LDA and QDA on all predictors; standardize + PCA ($d = 1-9$) + LDA or QDA; and a blend $P = wP_{\text{LDA}} + (1 - w)P_{\text{QDA}}$, $w = 0, 0.1, \dots, 1$, all with empirical priors and preprocessing fitted within folds. Each was scored by misclassification rate on the same stratified 5 training folds; we chose the lowest CV error, preferring the fewest parameters within one SE. The test set was used once, after selection.

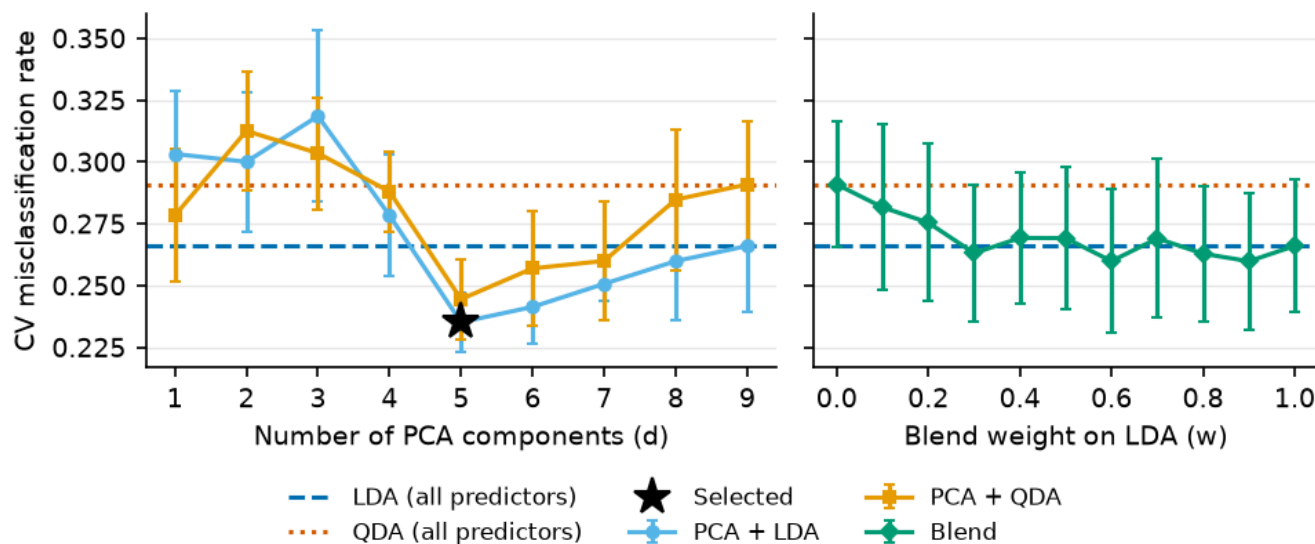


Figure 1: CV error (mean $\pm$ 1 SE); star marks the selected model.

Model	Best setting	CV error	SE
LDA	all predictors	0.266	0.027
QDA	all predictors	0.291	0.025
PCA + LDA	$d = 5$	0.235	0.012
PCA + QDA	$d = 5$	0.245	0.016
Blend	$w = 0.9$	0.260	0.028

Results. PCA + LDA ($d = 5$) had the lowest CV error and was the simplest model within one SE. Refitted on all training data, its **test misclassification rate is 0.338** (plain LDA: 0.324, plain QDA: 0.317, for reference only). Confusion matrix (rows observed, columns predicted): no CHD 70 / 21; CHD 26 / 22.

Findings and limitations.

- QDA trailed LDA and the best blend is 90% LDA: a shared covariance seems adequate.
- Five principal components cut CV error by about 0.03 versus LDA, roughly one SE: a modest gain.
- Test error exceeds CV error (choosing among 31 makes CV optimistic); with $SE \approx 0.04$ on 139 cases, it matches plain LDA/QDA within noise.
- Only 22 of 48 test CHD cases were detected, as misclassification rate weighs both errors equally. No fits failed.

(c) **File size.** `SKILL.md` is 4,314 bytes, under the 5,000-byte limit.

Explicit invocation. The skill is stored at `.claude/skills/lda-qda-report/SKILL.md` and used with Claude Code. Its frontmatter sets `disable-model-invocation: true`, so Claude never loads the skill on its own based on the conversation. It runs only when I type the slash command `/lda-qda-report` followed by my request. The skill's first instruction repeats this rule, and it asks for the dataset, response and predictors to be supplied at invocation, so the file itself contains nothing specific to SAheart.

Prompt used to apply the skill.

```
/lda-qda-report Analyze homework/week-07/data/saheart.csv. Response: chd (binary, 0/1). Predictors: all nine other variables as numeric (sbp, tobacco, ldl, adiposity, famhist, typea, obesity, alcohol, age), coding famhist as 1 = Absent, 2 = Present. Seek the best predictive performance among the LDA/QDA methods allowed by the skill, including ordinary LDA and QDA as comparisons. Use seed 4323. Save the script, figure, report .qmd and PDF in homework/week-07/q4/.
```

Review. Before applying the skill, I changed the reserved test set from 25% to 30% of the data. After the run, I checked that the test set was held out before any model choice, that all candidates used the same five training folds, that scaling and PCA were fitted inside each fold, and that every number in the report matches the script output. I did not revise the skill after seeing the results. The selected model (PCA + LDA, $d = 5$) had a higher test error than plain LDA and QDA, but changing the skill because of test-set performance would break the rule that model choices use only training and cross-validation results.